



BOOTCAMP FRONTEND & REACT · SEMANA 3

EL DOM

Clase 9 — La tienda reacciona: render desde el array, eventos y localStorage

4 h 15 · L·M·V

Proyecto TechCart

STEERCLAVE

PRIMERO CERRAMOS LA CLASE 8 · DESPUÉS, EL DOM

Agenda

1. Dónde quedamos + spread

16'

el ejercicio que faltó

2. Cierre de ES6+

30'

abreviadas · defaults · find · ?. · ??

3. Módulos

44'

export / import · 5 archivos

4. El DOM

40'

querySelector · pintar el catálogo

5. Eventos

46'

delegación · dataset

6. localStorage

34'

que sobreviva al refresco

7. Taller

26'

la lista del carrito y quitar

8. Cierre + 5 tareas

19'

esta vez sí

 **Receso de 15 min** entre el bloque 3 y el 4 · los bloques con franja naranja **cierran la Clase 8**

Dos por el chat

En `const { categoria: rubro } = p`, ¿cuál de los dos nombres tuvo que existir?

¿Por qué `[...productos].sort(...)` y no `productos.sort(...)`?

TRES DETALLES CORTOS QUE APARECEN EN TODO EL CÓDIGO MODERNO

Abreviadas, objeto y defaults

`{ cantidad, total }`

Propiedades **abreviadas**: si la propiedad se llama igual que la variable, se escribe una vez.

```
// antes
{ cantidad: cantidad }
// ahora
{ cantidad }
```

`p => ({ ... })`

Arrow que devuelve un **objeto**: va envuelto en **paréntesis**.

```
p => { etiqueta: p.nombre
} ✗ undefined
p => ({ etiqueta:
p.nombre }) ✓
```

`(precio, pct = 10)`

Parámetros **por defecto**.  Java lo resuelve con **sobrecarga**.

```
conDescuento(1000) // 900
conDescuento(1000, 25) //
750
```

🔗 EL NULLPOINTEREXCEPTION DE JAVASCRIPT, Y SUS DOS REDES

find · ?. · ??

```
const x = productos.find(p => p.nombre === "Switch")
x // undefined ← find no encontró nada
x.precio // ✗ TypeError · el script SE DETIENE
x?.precio // undefined, sin error
x?.precio ?? 0 // 0
```

stock || "sin dato"

Con stock: 0 da "sin dato". ✗

stock ?? "sin dato"

Da 0. Solo salta con null/undefined. ✓

find

devuelve **el primer** elemento que cumple... o undefined

🐞 **Hoy mismo:** uno de nuestros productos —el **Apple Watch**— no tiene imagen. Sin protección no rompe una tarjeta: **rompe todo el catálogo**, porque el script se detiene.

LA REGLA QUE YA CONOCEN DE JAVA: UNA CLASE, UN ARCHIVO

Un archivo, **una responsabilidad**

datos.js

el catálogo. Única
fuente de verdad.

carrito.js

el dinero: IGV,
resumen, descuentos.

formato.js

un solo trabajo:
precios listos para
mostrar.

ui.js

cómo se ve un
producto. No calcula.

main.js

orquesta: pide,
escucha, guarda y
pinta.



Caso real: el archivo revuelto no se queda en 100 líneas, llega a 2000. Y pasa algo peor que la incomodidad: **nadie lo quiere tocar**, porque cambiar el IGV significa entrar donde también viven la búsqueda, el render y la sesión. Ese archivo se vuelve intocable y el equipo empieza a copiar funciones a otros lados. Partirlo hoy cuesta diez minutos.

EXPORT = PUBLIC · LO QUE NO SE EXPORTA SE QUEDA ADENTRO DEL ARCHIVO

export / import

```
// js/datos.js
export const productos = [ ... ]

// js/main.js - con llaves, por nombre
import { productos } from "./datos.js"

// default: UNO por archivo, sin llaves
export default formatearPrecio
import formatearPrecio from "./formato.js"
```

```
<!-- index.html: el interruptor -->
<script type="module" src="js/main.js"></script>
```

los 3 errores que van a ver hoy

Cannot use import statement... falta type="module"

404 / Failed to resolve falta la extensión .js

does not provide an export... falta el export

 **El traicionero:** "./Datos.js" con mayúscula funciona en Windows y da 404 en Vercel (Linux). Nombres de archivo en minúscula, siempre.

EL NAVEGADOR CONVIERTE TU HTML EN UN ÁRBOL DE OBJETOS

La página también es un **objeto**

```
document
└─ html → body
    └─ header → h1
    └─ aside.lateral
    └─ main
        └─ section#catalogo
            └─ div.productos
                └─ article.card
                └─ article.card
```

```
document.querySelector("h1")
document.querySelectorAll(".card")

// y se modifica en vivo:
...querySelector("h1").textContent = "Ya
reacciona"
```

Los selectores son **los de CSS**: lo que aprendieron para **estilar**, hoy sirve para **buscar**.

 Si no encuentra nada devuelve **null**, no un error. El `Cannot read properties of null` llega en la línea siguiente, y la causa casi siempre es un **typo en el selector**.

LA PLANTILLA DE LA CLASE 8, AHORA CON ETIQUETAS HTML ADENTRO

innerHTML: JavaScript construye la tarjeta

```
export const tarjetaProducto = ({ id, nombre,
precio, stock, imagen }) => `
  <article class="card group" data-id="${id}">
    ${imagen ? `` : `<div>📦
  </div>`}
    <h3>${nombre}</h3>
    <strong>${formatearPrecio(precio)}</strong>
    <button data-accion="agregar" data-id="${id}">
      ${stock > 0 ? "Agregar al carrito" :
"Agotado"}
    </button>
  </article>`
```

TEXTCONTENT VS INNERHTML

textContent: solo texto, no interpreta nada.

innerHTML: lo interpreta como HTML... y **reemplaza todo** lo de adentro.

🦋 **XSS:** si el texto vino de **afuera** (un formulario, una API, un comentario) e incluye etiquetas, el navegador **las ejecuta**. Si es solo texto, usen **textContent**.

EL CATÁLOGO DEJA DE VIVIR EN EL HTML Y VIVE EN EL DATO

5 productos → 5 tarjetas

ANTES • INDEX.HTML

```
<article class="card">...</article>
<article class="card">...</article>
<article class="card">...</article>
<article class="card">...</article>
// 40 líneas escritas a mano
```



AHORA • JS/MAIN.JS

```
const pintarCatalogo = () => {
  contenedor.innerHTML = productos
    .map(tarjetaProducto)
    .join("")
}
```

map da un **array** de textos; innerHTML quiere **uno**. Eso los pega join.

LA FUNCIÓN NO CORRE AHORA: QUEDA ESPERANDO

addEventListener

```
titulo.addEventListener("click", (evento) => {  
  console.log(evento.type) // "click"  
  console.log(evento.target) // DÓNDE cayó el clic  
})
```

A quién le hablo · qué escucho · qué hago.  Java: su `addActionListener`.

click

botones

input

mientras el usuario escribe

submit

formularios

change

selects y checkboxes

LOS ELEMENTOS QUE JAVASCRIPT CREA NACEN SIN LISTENER

Delegación de eventos

```
// ✗ un listener por botón
document.querySelectorAll("button").forEach(b =>
  b.addEventListener("click", ...))
// funciona... hasta que repintas: los botones
// viejos MUEREN y los nuevos no tienen listener
```

```
// ✓ UN listener en el contenedor, que no se
  repinta
contenedor.addEventListener("click", (evento) => {
  const boton = evento.target.closest("button[data-
  accion='agregar']")
  if (!boton) return // el portero
  ...
})
```

El evento **sube** por el árbol hasta el contenedor. `closest` sube desde donde cayó el clic buscando el ancestro que coincide.

 **Caso real:** se reporta siempre con la misma frase — **"el botón funciona la primera vez y después no"**. O "funciona hasta que filtro por categoría", porque filtrar repinta. Y volver a enganchar listeners tras cada render crea el bug siguiente: **se agrega dos o tres veces por clic**.

¿CUÁL PRODUCTO ME HICIERON CLIC?

dataset: los datos van en el elemento

```
// en el HTML que genera ui.js
<button data-accion="agregar" data-id="4">

// desde JS
boton.dataset.accion // "agregar"
boton.dataset.id // "4" ← ¡TEXTO!

const id = Number(boton.dataset.id)
const producto = productos.find(p => p.id === id)
carrito = [...carrito, producto] // spread, no
push
```

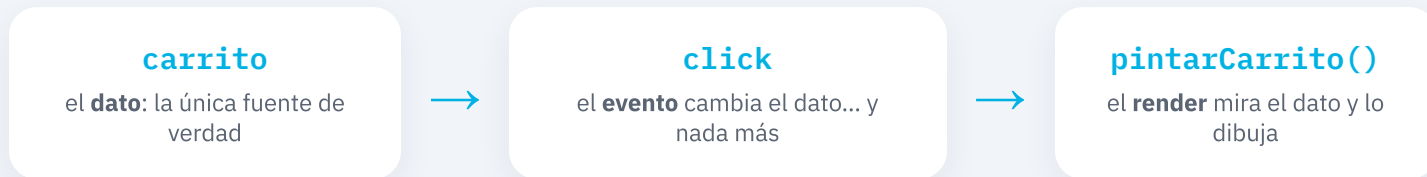
p.id === "4"

Sin `Number()`: `===` compara **valor y tipo**, así que es **false** para todos y `find` no encuentra nada.

 **Caso real:** "hago clic y no agrega nada, pero **no hay ningún error** en consola". No hay error porque `find` devolvió `undefined` tranquilamente. Todo lo que sale del HTML es **texto** hasta que ustedes lo conviertan.

SI SE LLEVAN UNA COSA DE HOY, QUE SEA ESTA

Un dato, una función que pinta



```
carrito = [...carrito, producto] // 1. cambio el dato
guardarCarrito(carrito) // 2. guardo
pintarCarrito() // 3. pinto
```

🐛 **Caso real:** cuando cada evento toca pedacitos de pantalla por su cuenta, la app termina **desincronizada**: el contador dice 3, la lista muestra 2 y el total no coincide con ninguno. Yo vi un carrito donde el precio del resumen y el del checkout no eran el mismo — dos lugares calculando por su lado.

SETITEM · GETITEM · REMOVEITEM —  UN MAP<STRING, STRING> QUE PERSISTE

localStorage solo guarda TEXTO

```
// ❌ lo que todos intentan primero
localStorage.setItem(CLAVE, carrito)
localStorage.getItem(CLAVE)
// "[object Object],[object Object]" ← basura
```

```
// ✅ siempre en pareja
localStorage.setItem(CLAVE,
JSON.stringify(carrito))
const guardado =
JSON.parse(localStorage.getItem(CLAVE))
```

stringify = de objeto a texto.

parse = de texto a objeto.

Y la clave, en una **constante**: escrita a mano en cuatro lugares, basta una letra distinta para "perder" el carrito sin ningún error.

 **Caso real:** `JSON.parse` **revienta** si el texto no es JSON válido (basura vieja, un guardado a medias). Y como corre al **arrancar**, la app **no abre** para ese usuario: pantalla en blanco imposible de reproducir en tu máquina.

TODO ACCESO AL ALMACENAMIENTO VA ENVUELTO

Cargar al arrancar, y **try/catch**

```
const cargarCarrito = () => {  
  try {  
    const crudo = localStorage.getItem(CLAVE)  
    return crudo ? JSON.parse(crudo) : []  
  } catch (error) {  
    console.warn("Carrito inválido, empiezo  
vacío")  
    return [] // la app ABRE, aunque pierda el  
carrito  
  }  
}  
  
let carrito = cargarCarrito()
```



Y al escribir también falla: `setItem` lanza excepción cuando el almacenamiento está **lleno** (~5 MB) y en **Safari en modo privado**. Sin `try/catch`, en un iPhone en navegación privada la tienda se cae al primer producto — y en tu máquina nunca lo vas a reproducir.

`sessionStorage`

la **misma API**, con una diferencia: se borra **al cerrar la pestaña**. Para lo que solo vale en esta visita.



Nunca contraseñas ni datos sensibles: cualquier JS de la página lo lee.

EL MISMO MOVIMIENTO DEL CATÁLOGO: DATO → HTML → MAP + JOIN

La lista del carrito, y **quitar**

```
// ui.js - recibe el producto Y SU POSICIÓN
export const filaCarrito = ({ nombre, precio },
indice) => `
  <li>... <button data-accion="quitar" data-
indice="${indice}">
  Quitar</button></li>`

// main.js
listaCarrito.innerHTML =
carrito.map(filaCarrito).join("")

carrito = carrito.filter((producto, i) => i !==
indice)
```

map le entrega a su función el elemento **y su posición**. Es la primera vez que usamos ese segundo parámetro.



¿Por qué quito por posición y no por id? Porque hoy el mismo producto puede entrar dos veces, y un filter por id borraría **las dos filas**. Es el caso real de la clase pasada: **el usuario quita un producto y desaparece otro**. Con cantidades (tarea difícil 2), vuelve a ser por id.

HOY LA TIENDA DEJÓ DE SER UNA FOTO

Resumen

querySelector

selectores de CSS · si no encuentra, null

textContent · innerHTML

texto vs etiquetas · reemplaza todo

map + join

el catálogo se pinta desde el array

addEventListener

delegación en el contenedor + closest

dataset

siempre texto → Number()

localStorage

JSON.stringify / parse + try/catch

LA DIFÍCIL 2 LA VAMOS A NECESITAR ARMADA PARA LAS CLASES DE REACT

5 tareas

FÁCIL

El contador en la pestaña: `document.title` con cuántos productos hay, desde `pintarCarrito`.

INTERMEDIO

Filtrar por categoría: los enlaces de la barra lateral, con delegación y `data-categoria`. Ojo: al filtrar se repinta.

INTERMEDIO

contarPorCategoria(items): un `reduce` con acumulador **objeto** $\rightarrow \{ \text{laptops: 1, audio: 2} \}$.

DIFÍCIL

Tema oscuro que se recuerda: `classList.toggle("dark")` + guardarlo en `localStorage`.

DIFÍCIL

Carrito con cantidades: cada producto entra una vez con cantidad; sube en vez de duplicar. Y quitar vuelve a ser por `id`.

Próxima clase

Los productos **de verdad**

Hay una mentira en el corazón de todo esto: los **cinco productos que escribí a mano**. Ninguna tienda funciona así — los productos viven en un **servidor**. La próxima clase los pedimos de verdad con **fetch**, y aparece un problema nuevo: los datos **no llegan al instante**. Hay que pedirlos, esperar y mostrarle algo al usuario mientras tanto. Eso es código **asíncrono**: promesas y **async / await**.

Espíenlo ahora: [abran dummyjson.com/products](https://dummyjson.com/products) y miren lo que devuelve.